

Introduction

Stacks provides library discovery and catalog browsing across multiple library systems. Users can search for books via the Open Library API, locate nearby libraries using geolocation, and browse pre-scraped catalogs from Houston Public Library and LA County Library.

In addition to discovery, Stacks enhances the reading experience through personalized and interactive features. Users can organize books into custom shelves, including a “Want to Read” list, and utilize an AI-powered Book Finder that presents saved books in a swipe-based interface to help decide what to read next. The application also promotes library usage by estimating the amount of money users save by borrowing books instead of purchasing them.

System Objectives

The Stacks application aims to:

1. Simplify Book Discovery: Search millions of books via Open Library API and locate availability in nearby libraries
2. Location-Based Library Search: Identify closest library branches with geolocation support
3. Catalog Browsing: Browse curated library catalogs by category
4. Intuitive Navigation: Category-based organization, alphabetized listings, and search bars
5. User Profiles: Save favorite items and track usage patterns
6. Accessibility & Responsiveness: Support all experience levels on desktop and mobile
7. AI-Powered Book Selection: Help users choose their next read using an interactive swipe-based recommendation system
8. TBR (To-Be-Read) Optimization: Encourage users to read books they already own or saved to shelf by surfacing personalized suggestions
9. Library Savings Awareness: Estimate and display money saved by borrowing instead of purchasing books
10. Expanded Item Discovery: Support adding books to shelves via search and integrating “Library of Things” items into browsing

Hardware, Software, and Human Interfaces

Browser Interface

- Supported: Chrome 90+, Firefox 88+, Safari 14+, Edge 90+

- APIs: Geolocation, Fetch, Local Storage

External APIs

- Open Library API: Search books (/search.json), book details (/works/{id}.json), covers
- Nominatim API: Forward/reverse geocoding (/search, /reverse), country filtering

Frameworks & Libraries

- React 18: Component-based UI rendering, state management
- React Router DOM 6: SPA navigation
- Vite 5: Fast dev server, hot module replacement

User Interface

- NavBar: Home, Discover, Browse, Libraries, Profile
- Main content: Card-based item/library listings, responsive design
- Font: Inter (UI), Crimson Pro (headings)
- Colors: Primary Blue (#3b82f6), text dark gray (#1a1a1a), background white/light gray (#f9fafb)

Hardware Interfaces

- Input: Mouse, keyboard, touch
- Display: 320x568 minimum, 1920x1080 recommended
- Geolocation: GPS/WiFi/IP-based with permission prompt

Architectural Design

The Stacks application follows SPA architecture with a React-adapted MVC pattern:

- Model: Data adapters and service modules
- View: React UI components
- Controller: State and event handlers

Navigation is handled client-side; external APIs are queried on demand.

Major Software Components (CSCs)

CSC	Purpose	Key Modules
Routing & Navigation	Manage app routes	App.jsx, Navigation.jsx, Footer.jsx

Page Components	Top-level views	Home.jsx, Discover.jsx, Results.jsx, Library.jsx, Libraries.jsx, CategoryItems.jsx, Item.jsx, Profile.jsx
Shared UI Components	Reusable UI elements	Navigation.jsx, Footer.jsx, BookCard.jsx, LibraryCard.jsx, SearchBar.jsx
Data Adapters & Services	API integration, data transformation	openlibrary.js, finder.js, geocoding.js, bookIdentity.js
Library Registry	Static library data	california-libraries.js, socal-libraries.js
Scraper Subsystem	Catalog data extraction	scraper/ (Python runtime)

Key Modules

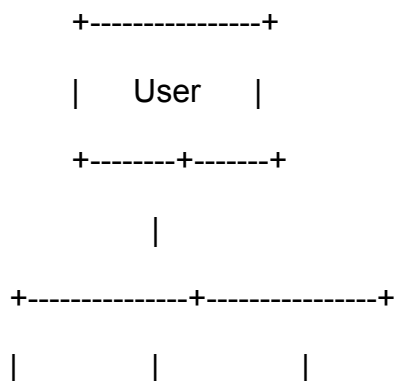
Book Finder (Swipe Engine)	AI-driven recommendation and swipe decision system	BookFinder.jsx, SwipeCard.jsx, recommendationEngine.js
Savings Tracker	Calculates estimated savings from borrowed books	savingsCalculator.js
Shelf Management System	Manage user book collections and statuses (e.g., TBR, read)	shelfManager.js
Library of Things Integration	Supports browsing and saving non-book borrowable items	thingsCatalog.js

Major Software Interactions

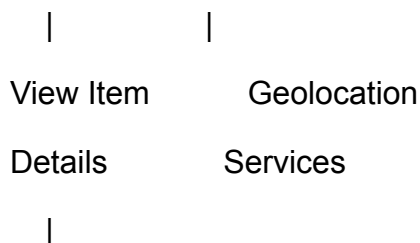
1. Page ↔ Data Adapters: Direct function calls (async), JavaScript objects, error handled via try/catch.
2. Adapters ↔ External APIs: HTTPS REST calls, JSON format, network errors handled with context.
3. Routing ↔ Page Components: React Router passing route parameters via useParams.
4. Discover Page ↔ Geolocation Services: Calls getCurrentLocation() and findNearbyLibraries(), fallback input if user denies permission.
5. Browser Storage ↔ Application State: localStorage for favorites/preferences; JSON serialization with error handling.
6. Book Finder ↔ Shelf Manager: Retrieves TBR books and updates preferences based on swipe actions
7. Book Finder ↔ Recommendation Engine: Sends user interaction data to refine suggestions
8. Savings Tracker ↔ Profile: Displays accumulated savings and updates when user marks a book as borrowed
9. Shelf Manager ↔ Local Storage: Persists user shelves and borrowing status
10. Library of Things ↔ Page Components: Displays items similarly to books within browsing interfaces

Architectural Diagrams

1. Use Case Diagram



Browse Catalog Search Books Find Nearby Libraries



Manage Profile

/ \

Save Favorites View History

+-----+

| Open Library API |

+-----+

^

|

Provides Book Data

+-----+

| Nominatim API |

+-----+

^

|

Provides Geocoding

2. Component Diagram

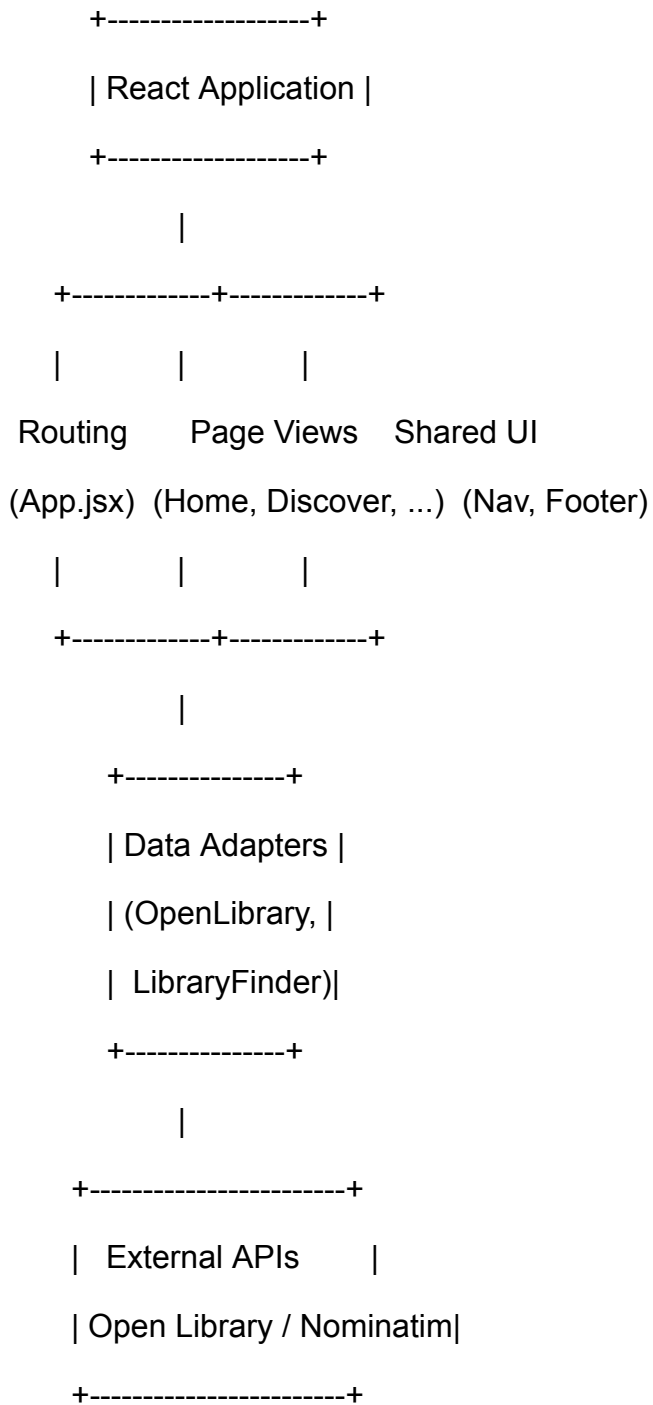
+-----+

| Browser UI |

+-----+

|

v



3. Class Diagram

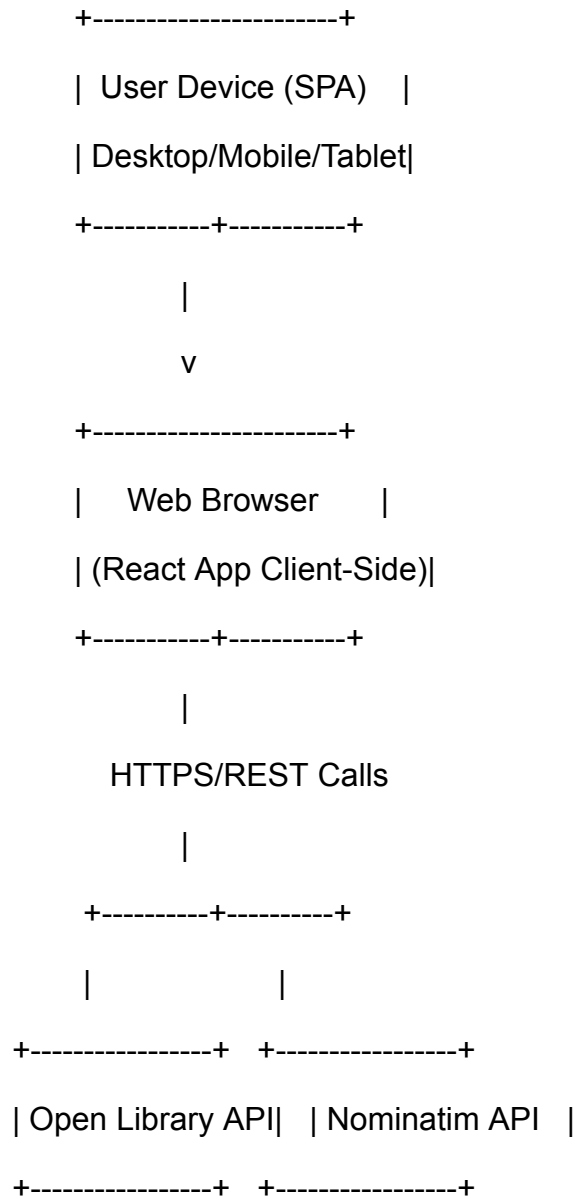
+-----+

```

| App.jsx |
| (Router) |
+-----+-----+
|
+-----+-----+
|      |      |
+-----+ +-----+ +-----+
| Navigation || Routes || Footer |
+-----+ +-----+ +-----+
|      |
|      +----+----+
|      | Pages |
|      | (Home, |
|      | Discover|
|      | Library)|
|      +----+----+
|      |
+-----+-----+
|
+-----+-----+
| Data Adapters|
| OpenLibrary, |
| LibraryFinder|
+-----+

```

4. Deployment Diagram: User Device (browser) ↔ HTTPS ↔ Web Server ↔ External APIs (Open Library, Nominatim).



CSC and CSU Descriptions

Routing and Navigation

- CSU: App.jsx: Manages routes, renders page components
- CSU: Navigation.jsx: Renders navigation bar

- CSU: Footer.jsx: Site footer

Page Components

- CSUs: Home.jsx, Discover.jsx, Results.jsx, Library.jsx, Libraries.jsx, CategoryItems.jsx, Item.jsx, Profile.jsx
- Responsibilities: Layout, data fetching, state management, user interactions

Data Adapters & Services

- CSU: OpenLibraryAdapter: Search books, get details, fetch covers
- CSU: LibraryFinder: Find nearby libraries, calculate distance, geocode location

Library Registry

- Stores static library metadata and coordinates for distance calculation

Scraper Subsystem

- Extracts, cleans, and exports library catalog data (Python runtime, separate from SPA)

Book Finder

- Displays swipe-based interface (similar to Tinder-style interaction) and pulls books from user's "Want to Read" shelf

Savings Tracker

- Estimates cost of books using Open Library metadata or average price assumptions and tracks whether a user borrowed a book from a library (user input). Aggregates total savings across user history

Shelf Management System

- Handles adding/removing books from shelves (Want to Read, Reading, Read, Didn't Finish) and stores on Firestore

Database Design and Description

Stacks uses a hybrid data approach, combining external APIs with a cloud-hosted database for user-specific data.

- Open Library API: Provides book search results, metadata, and covers
- Nominatim API: Provides geolocation and reverse geocoding data
- Pre-scraped JSON catalogs for supported libraries

Stacks uses Cloud Firestore as a NoSQL database to store and manage user-specific data in real time.

Collections Structure

- users
 - userId (document ID)
 - name
 - email
 - createdAt
- shelves
 - userId
 - wantToRead: Book
 - read: Book
 - reading: Book

User authentication is handled through Firebase Authentication. It supports email/password and google sign in

Firestore security rules restrict access to authenticated users only so users can only read/write their own data (based on userId)